

Задание 1

- 1. Изучите проект.
- 2. Инициализируйте проект, выполните код.

Разбор решения задачи 1

Приложите скриншот входящих правил «Группы безопасности» в ЛК Yandex Cloud .

cloud-kost1n-valery

default

Virtual Private Cloud / Облачные сети / develop

develop

Облачная сеть

Обзор

Таблицы маршрутизации

Группы безопасности

DNS зоны

Операции

Группы безопасности

Фильтр по имени или идентификатору

Имя ↑↓	Идентификатор ↑↓	Описание	Статус	Правила	Дата создания ↑↓	Метки	
default-sg-enp2f193jhjbc9r98ov	Default enpg9mpjjseq19p3htaj	Default security group for network	Active	2	21.05.2026, в 21:09	—	...
example_dynamic	enpmttnpva47uq91qaee	—	Active	4	21.05.2026, в 21:09	—	...

default-sg-en... Default
Группа безопасности

Обзор

Обзор

Общее

Идентификатор enpg9mpjjseq19p3htaJ
Статус Active
Имя default-sg-enp2f193jhjibc9r98ov
Описание Default security group for network
Сеть develop

Правила

Входящий трафик Исходящий трафик

Протокол	Диапазон портов	Тип источника	Источник	Описание
Any	—	CIDR	0.0.0.0/0	—

example_dynamic
Группа безопасности

Обзор

Обзор

Общее

Идентификатор..... enpmttnpva47uq91qaae

Статус..... Active

Имя..... example_dynamic

Сеть..... develop

Правила

Входящий трафик

Исходящий трафик

Протокол	Диапазон портов	Тип источника	Источник	Описание
TCP	22	CIDR	0.0.0.0/0	разрешить входящий ssh
TCP	80	CIDR	0.0.0.0/0	разрешить входящий http
TCP	443	CIDR	0.0.0.0/0	разрешить входящий https

Задание 2

1. Создайте файл count-vm.tf. Опишите в нём создание двух **одинаковых** VM web-1 и web-2 (не web-0 и web-1) с минимальными параметрами, используя мета-аргумент **count loop**. Назначьте VM созданную в первом задании группу безопасности.(как это сделать узнайте в документации провайдера yandex/compute_instance)
2. Создайте файл for_each-vm.tf. Опишите в нём создание двух VM для баз данных с именами "main" и "replica" **разных** по cpu/ram/disk_volume , используя мета-аргумент **for_each loop**.

Используйте для обеих VM одну общую переменную типа:

```
1 variable "each_vm" {
2     type = list(object({ vm_name=string, cpu=number, ram=number, disk_volume=number
3     }))
4 }
```

При желании внесите в переменную все возможные параметры.

3. VM, описанные в файле count-vm.tf, должны создаваться после VM, описанных в файле for_each-vm.tf.

4. Используйте функцию file в local-переменной для считывания ключа ~/.ssh/id_rsa.pub и его последующего использования в блоке metadata, взятому из ДЗ 2.

5. Инициализируйте проект, выполните код.

Разбор решения задачи 2

Добавил `locals` с SSH-ключом в `main.tf`, а также значение для образа VM. Итоговое содержание файла:

```
1 resource "yandex_vpc_network" "develop" {
2     name = var.vpc_name
3 }
4 resource "yandex_vpc_subnet" "develop" {
5     name          = var.vpc_name
```

```
6     zone          = var.default_zone
7     network_id     = yandex_vpc_network.develop.id
8     v4_cidr_blocks = var.default_cidr
9 }
10
11 locals {
12     ssh_key = file("~/ssh/id_ed25519.pub")
13 }
14
15 data "yandex_compute_image" "ubuntu" {
16     family = "ubuntu-2204-lts"
17 }
```

Создал файл `count-vm.tf` следующего содержания:

```
1 resource "yandex_compute_instance" "web" {
2     count = 2
3
4     name          = "web-${count.index + 1}"
5     platform_id   = "standard-v1"
6     zone          = var.default_zone
7
8     depends_on = [yandex_compute_instance.db]
9 }
```

```
10 resources {
11     cores          = 2
12     memory         = 1
13     core_fraction = 5
14 }
15
16 boot_disk {
17     initialize_params {
18         image_id = data.yandex_compute_image.ubuntu.id
19         size     = 8
20     }
21 }
22
23 network_interface {
24     subnet_id      = yandex_vpc_subnet.develop.id
25     nat            = true
26     security_group_ids = [yandex_vpc_security_group.example.id]
27 }
28
29 metadata = {
30     ssh-keys = "ubuntu:${local.ssh_key}"
31 }
32 }
```

Из содержимого файла:

- `count = 2` - создаёт 2 копии ресурса
- `name = "web- $\{\text{count.index} + 1\}$ "` - имена будут начинаться не с 0: `web-1` и `web-2`
- `depends_on = [yandex_compute_instance.db]` - порядок создания - ссылка на ресурс из `for_each-vm.tf`,
- `security_group_ids = [yandex_vpc_security_group.example.id]` -назначает группу безопасности `example`, созданную в первом задании
- `ssh-keys = "ubuntu: $\{\text{local.ssh\_key}\}$ "` - SSH-ключ из `locals`
- Задал размер диска 8 гб, минимально необходимые для образа VM

Добавил переменную `each_vm` в `variables.tf`. Итоговое содержание файла:

```
1  ###cloud vars
2      # variable "token" {
3      #   type          = string
4      #   description = "OAuth-token;
5      #               https://cloud.yandex.ru/docs/iam/concepts/authorization/oauth-token
6      # }
7
8      variable "cloud_id" {
9          type          = string
10         default       = "b1g3g3a6ro9rr9hd0mvm"
```

```
10     description = "https://cloud.yandex.ru/docs/resource-
manager/operations/cloud/get-id"
11 }
12
13 variable "folder_id" {
14     type          = string
15     default       = "b1grsjmidldjs2rmkgd0"
16     description = "https://cloud.yandex.ru/docs/resource-
manager/operations/folder/get-id"
17 }
18
19 variable "default_zone" {
20     type          = string
21     default       = "ru-central1-a"
22     description = "https://cloud.yandex.ru/docs/overview/concepts/geo-scope"
23 }
24 variable "default_cidr" {
25     type          = list(string)
26     default       = ["10.0.1.0/24"]
27     description = "https://cloud.yandex.ru/docs/vpc/operations/subnet-create"
28 }
29
30 variable "vpc_name" {
31     type          = string
```



```
32     default      = "develop"
33     description = "VPC network&subnet name"
34 }
35
36 variable "each_vm" {
37     type = list(object({
38         vm_name      = string
39         cpu           = number
40         ram           = number
41         disk_volume  = number
42     }))
43     default = [
44         {
45             vm_name      = "main"
46             cpu           = 4
47             ram           = 4
48             disk_volume  = 10
49         },
50         {
51             vm_name      = "replica"
52             cpu           = 2
53             ram           = 2
54             disk_volume  = 8
55         }
56     ]
57 }
```

```
56     ]
57   }
58
```

Здесь задал параметры каждой VM. Заменяю размер диска на мин. 8 гб, необходимые для образа VM.

Создал файл `for_each-vm.tf`:

```
1  resource "yandex_compute_instance" "db" {
2
3    for_each = { for vm in var.each_vm : vm.vm_name => vm }
4
5    name          = each.value.vm_name
6    platform_id   = "standard-v1"
7    zone          = var.default_zone
8
9    resources {
10      cores          = each.value.cpu
11      memory          = each.value.ram
12      core_fraction  = 20
13    }
14
15    boot_disk {
```

```
16     initialize_params {
17         image_id = data.yandex_compute_image.ubuntu.id
18         size      = each.value.disk_volume
19     }
20 }
21
22 network_interface {
23     subnet_id = yandex_vpc_subnet.develop.id
24     nat        = true
25 }
26
27 metadata = {
28     ssh-keys = "ubuntu:${local.ssh_key}"
29 }
30 }
```

В созданном файле:

- `for_each = { for vm in var.each_vm : vm.vm_name => vm }`

это выражение преобразует список объектов в `map` вида `...{"main" = {...}, "replica" = {...}}...`. Terraform требует `map` или `set` для `for_each`, а поскольку задан список (list), необходимо преобразование (источник:

https://developer.hashicorp.com/terraform/language/meta-arguments/for_each,

<https://developer.hashicorp.com/terraform/language/expressions/for>,
<https://developer.hashicorp.com/terraform/language/meta-arguments/count>).

- `each.value.vm_name` , `each.value.cpu` это обращение к полям текущего объекта итерации.
- BM `main` получит 4 CPU / 4GB RAM / 10GB диск, `replica` получит 2CPU/2GB RAM/8GB, то есть разные параметры через одну переменную
- `platform_id = "standard-v1"` - платформа Intel Xeon E5-2660 v4 (Broadwell), которая поддерживает `core_fraction = 5` (5% от ядра), максимально дешевая BM
- `image_id` из переменной образа BM в файле `main.tf`

Иницирую проект командами:

```
1 terraform init
2 terraform validate
3 terraform fmt
4 terraform plan
5 terraform apply
```

```
• vboxuser@KVA:~/terraform/hw-03$ terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of yandex-cloud/yandex from the dependency lock file
- Using previously-installed yandex-cloud/yandex v0.204.0
```

Initializing the backend...

Initializing provider plugins found in the state...

- Reusing previous version of yandex-cloud/yandex
- Using previously-installed yandex-cloud/yandex v0.204.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
• vboxuser@KVA:~/terraform/hw-03$ terraform apply
yandex_vpc_network.develop: Refreshing state... [id=enp2fl93jhjibc9r98ov]
data.yandex_compute_image.ubuntu: Reading...
data.yandex_compute_image.ubuntu: Read complete after 0s [id=fd85f16aijlrvpce9gas]
yandex_vpc_subnet.develop: Refreshing state... [id=e9bv4rbjflbas3mbvqh8]
yandex_vpc_security_group.example: Refreshing state... [id=enpmttnpva47uq9lqaae]
yandex_compute_instance.db["replica"]: Refreshing state... [id=fhmeoe671732kedf7e9g]
yandex_compute_instance.db["main"]: Refreshing state... [id=fhmdjppq47damfeibel9m]
yandex_compute_instance.web[1]: Refreshing state... [id=fhmpkpppm6crmpmfvqca]
yandex_compute_instance.web[0]: Refreshing state... [id=fhm5hr3sf0vbh6ele5qg]
```

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

```
• vboxuser@KVA:~/terraform/hw-03$
```

Виртуальные машины

Фильтр по имени или ID Все статусы Все платформы Все зоны доступности Требуется обслуживание

☐ Имя ↑↓	Статус ↑↓	ОС	Публичный IPv4	Идентификатор ↑↓	RAM	vCPU	Размер дисков	Метки	Cloud Backup	Платформа ↑↓	Доля vCPU	Прерываемая	Зона доступности ↑↓	Внутренний IPv4	Дата создания ↑↓	Группа размещения	Раздел (partition)	Пул резервов ВМ ⓘ	⚙
☐ web-1	Running		89.169.143.159	fhm5hr3sf8vbnhe1e5qg	1 Гб	2	8 Гб	—	Не подключён	Intel Broadwell	5%	Нет	ru-central1-a	10.0.1.23	22.05.2026, в 22:06	—	—	—	...
☐ main	Running		84.252.131.99	fhmdjq47damfe1be19m	4 Гб	4	10 Гб	—	Не подключён	Intel Broadwell	20%	Нет	ru-central1-a	10.0.1.32	22.05.2026, в 22:00	—	—	—	...
☐ replica	Running		111.88.244.132	fhmeoe671732kedf7e9g	2 Гб	2	8 Гб	—	Не подключён	Intel Broadwell	20%	Нет	ru-central1-a	10.0.1.34	22.05.2026, в 22:03	—	—	—	...
☐ web-2	Running		93.77.182.170	fhnpkppp6csmpefvqca	1 Гб	2	8 Гб	—	Не подключён	Intel Broadwell	5%	Нет	ru-central1-a	10.0.1.15	22.05.2026, в 22:06	—	—	—	...

Запустил в гит репозиторий

Задание 3

1. Создайте 3 одинаковых виртуальных диска размером 1 Гб с помощью ресурса `yandex_compute_disk` и мета-аргумента `count` в файле **disk_vm.tf**.
2. Создайте в том же файле **одиночную** (использовать `count` или `for_each` запрещено из-за задания №4) ВМ с именем "storage". Используйте блок **dynamic secondary_disk{..}** и мета-аргумент `for_each` для подключения созданных вами дополнительных дисков.

Разбор решения задачи 3

ссылка на документацию: https://yandex.cloud/en/docs/terraform/resources/compute_disk,
<https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks>

Создал файл `disk_vm.tf` со следующим содержимым:

```
1 resource "yandex_compute_disk" "disk" {
2     count = 3
3
4     name = "disk-${count.index + 1}"
5     type = "network-hdd"
6     zone = var.default_zone
7     size = 1
8 }
9
10 resource "yandex_compute_instance" "storage" {
11     name          = "storage"
12     platform_id   = "standard-v1"
13     zone          = var.default_zone
14
15     resources {
16         cores      = 2
17         memory     = 1
18         core_fraction = 5
19     }
20
21     boot_disk {
22         initialize_params {
23             image_id = data.yandex_compute_image.ubuntu.id
24             size      = 8
25         }
26     }
27 }
```

```
25     }
26 }
27
28 network_interface {
29     subnet_id      = yandex_vpc_subnet.develop.id
30     nat            = true
31     security_group_ids = [yandex_vpc_security_group.example.id]
32 }
33
34 metadata = {
35     ssh-keys = "ubuntu:${local.ssh_key}"
36 }
37
38 dynamic "secondary_disk" {
39     for_each = yandex_compute_disk.disk
40     content {
41         disk_id = secondary_disk.value.id
42     }
43 }
44 }
```

`count = 3` создаёт три одинаковых диска, `name = "disk-${count.index + 1}"` даёт имена `disk-1`, `disk-2`, `disk-3`
`secondary_disk` это вложенный блок внутри `yandex_compute_instance`, для него нельзя

написать обычный `for_each` на уровне ресурса; для таких случаев нужен `dynamic`. При этом в `dynamic` можно передавать коллекции напрямую, и он создаёт столько вложенных блоков, сколько элементов в коллекции.

Внутри `dynamic "secondary_disk"` выражение `for_each = yandex_compute_disk.disk` проходит по всем экземплярам дисков, созданным через `count`, а `secondary_disk.value.id` подставляет `id` каждого диска в соответствующий вложенный блок `secondary_disk`

```
• vboxuser@KVA:~/terraform/hw-03$ terraform validate
Success! The configuration is valid.
```

```
• vboxuser@KVA:~/terraform/hw-03$ terraform plan
yandex_vpc_network.develop: Refreshing state... [id=enp2fl93jhjibc9r98ov]
data.yandex_compute_image.ubuntu: Reading...
data.yandex_compute_image.ubuntu: Read complete after 0s [id=fd85f16aijlrvpce9gas]
yandex_vpc_subnet.develop: Refreshing state... [id=e9bv4rbjflbas3mbvqh8]
yandex_vpc_security_group.example: Refreshing state... [id=enpmttnpva47uq9lqaae]
yandex_compute_instance.db["main"]: Refreshing state... [id=fhmdjppq47damfeibel9m]
yandex_compute_instance.db["replica"]: Refreshing state... [id=fhmeoe671732kedf7e9g]
yandex_compute_instance.web[0]: Refreshing state... [id=fhm5hr3sf0vbh6ele5qg]
yandex_compute_instance.web[1]: Refreshing state... [id=fhmpkpppm6crmpmfvqca]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# yandex_compute_disk.disk[0] will be created
+ resource "yandex_compute_disk" "disk" {
  + block_size = 4096
  + created_at = (known after apply)
  + folder_id  = (known after apply)
  + id         = (known after apply)
  + name       = "disk-1"
  + product_ids = (known after apply)
  + size       = 1
  + status     = (known after apply)
  + type       = "network-hdd"
  + zone       = "ru-central1-a"

  + disk_placement_policy (known after apply)
  + hardware_generation (known after apply)
}
```

```
# yandex_compute_disk.disk[1] will be created
+ resource "yandex_compute_disk" "disk" {
  + block_size = 4096
  + created_at = (known after apply)
  + folder_id  = (known after apply)
  + id         = (known after apply)
  + name       = "disk-2"
  + product_ids = (known after apply)
  + size       = 1
  + status     = (known after apply)
  + type       = "network-hdd"
  + zone       = "ru-central1-a"

  + disk_placement_policy (known after apply)

  + hardware_generation (known after apply)
}

# yandex_compute_disk.disk[2] will be created
+ resource "yandex_compute_disk" "disk" {
  + block_size = 4096
  + created_at = (known after apply)
  + folder_id  = (known after apply)
  + id         = (known after apply)
  + name       = "disk-3"
  + product_ids = (known after apply)
  + size       = 1
  + status     = (known after apply)
  + type       = "network-hdd"
  + zone       = "ru-central1-a"

  + disk_placement_policy (known after apply)

  + hardware_generation (known after apply)
}

# yandex_compute_instance.storage will be created
+ resource "yandex_compute_instance" "storage" {
  + created_at      = (known after apply)
  + folder_id       = (known after apply)
  + fqdn            = (known after apply)
  + gpu_cluster_id  = (known after apply)
```

```

+ hardware_generation = (known after apply)
+ hostname            = (known after apply)
+ id                  = (known after apply)
+ maintenance_grace_period = (known after apply)
+ maintenance_policy   = (known after apply)
+ metadata             = {
  + "ssh-keys" = <<-EOT
    ubuntu:ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICGcxz6Ttx4jaiMvpdANwL4va+6RTIWd+Vd8yLDNu1xD vboxuser@KVA
  EOT
}
+ name = "storage"
+ network_acceleration_type = "standard"
+ platform_id = "standard-v1"
+ status = (known after apply)
+ zone = "ru-central1-a"

+ boot_disk {
  + auto_delete = true
  + device_name = (known after apply)
  + disk_id = (known after apply)
  + mode = (known after apply)

  + initialize_params {
    + block_size = (known after apply)
    + description = (known after apply)
    + image_id = "fd85f16aijlrvpce9gas"
    + name = (known after apply)
    + size = 8
    + snapshot_id = (known after apply)
    + type = "network-hdd"
  }
}

+ metadata_options (known after apply)

+ network_interface {
  + index = (known after apply)
  + ip_address = (known after apply)
  + ipv4 = true
  + ipv6 = (known after apply)
  + ipv6_address = (known after apply)
  + mac_address = (known after apply)
}

```

```

+ mac_address      = (known after apply)
+ nat              = true
+ nat_ip_address   = (known after apply)
+ nat_ip_version   = (known after apply)
+ security_group_ids = [
  + "enpmttnpva47uq9lqaee",
]
+ subnet_id        = "e9bv4rbjflbas3mbvqh8"
}

+ placement_policy (known after apply)

+ resources {
  + core_fraction = 8
  + cores         = 2
  + memory        = 1
}

+ scheduling_policy (known after apply)

+ secondary_disk {
  + auto_delete = false
  + device_name = (known after apply)
  + disk_id     = (known after apply)
  + mode        = "READ_WRITE"
}
+ secondary_disk {
  + auto_delete = false
  + device_name = (known after apply)
  + disk_id     = (known after apply)
  + mode        = "READ_WRITE"
}
+ secondary_disk {
  + auto_delete = false
  + device_name = (known after apply)
  + disk_id     = (known after apply)
  + mode        = "READ_WRITE"
}
}

```

Plan: 4 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

vboxuser@KVA:~/terraform/hw-03\$

Виртуальные машины

Фильтр по имени или ID Все статусы Все платформы Все зоны доступности Требуется обслуживание

Имя ↑	Статус ↑	ОС	Публичный IPv4	Идентификатор ↑	RAM	vCPU	Размер дисков	Метки	Cloud Backup	Платформа ↑	Доля vCPU	Прерываемая	Зона доступности ↑	Внутренний IPv4	Дата создания ↑	Группа размещения	Раздел (partition)	Пул резервов VM	
web-1	Running		89.169.143.159	fhm5hr3sf8vvh6e1e5qg	1 ГБ	2	8 ГБ		Не подключён	Intel Broadwell	5%	Нет	ru-central1-a	10.0.1.23	22.05.2026, в 22:06	—	—	—	...
storage	Running		111.88.240.200	fhm9utk3qpsncc3rhbk	1 ГБ	2	11 ГБ		Не подключён	Intel Broadwell	5%	Нет	ru-central1-a	10.0.1.36	22.05.2026, в 22:28	—	—	—	...
main	Running		84.252.131.99	fhdjpa47damfe1be19m	4 ГБ	4	10 ГБ		Не подключён	Intel Broadwell	20%	Нет	ru-central1-a	10.0.1.32	22.05.2026, в 22:00	—	—	—	...
replica	Running		111.88.244.132	fhmeoe671732kedf7e9g	2 ГБ	2	8 ГБ		Не подключён	Intel Broadwell	20%	Нет	ru-central1-a	10.0.1.34	22.05.2026, в 22:03	—	—	—	...
web-2	Running		93.77.182.170	fhmpppm6cmfpmfvqca	1 ГБ	2	8 ГБ		Не подключён	Intel Broadwell	5%	Нет	ru-central1-a	10.0.1.15	22.05.2026, в 22:06	—	—	—	...

Диски и хранилища

Диски Файловые хранилища

Фильтр по имени Все статусы Все типы Все зоны доступности

☐ Имя ↑		Тип ↑↓	Размер ↑↓	Статус ↑↓	Зашифрован	Виртуальные машины	Зона доступности ↑↓	Дата создания ↑↓	Группа размещения	Идентификатор ↑↓	Метки	⚙
☐ fhm1bhvhkgchtfhokjvk	Подключен	HDD	8 ГБ	Ready	Нет	storage	ru-central1-a	22.05.2026, в 22:28	—	fhm1bhvhkgchtfhokjvk	—	...
☐ fhmaq34f6so0htuir71a	Подключен	HDD	8 ГБ	Ready	Нет	web-2	ru-central1-a	22.05.2026, в 22:06	—	fhmaq34f6so0htuir71a	—	...
☐ fhmj4v5cvuoqumg6mmo	Подключен	HDD	10 ГБ	Ready	Нет	main	ru-central1-a	22.05.2026, в 22:01	—	fhmj4v5cvuoqumg6mmo	—	...
☐ fhmproud00mrurrd2m3d	Подключен	HDD	8 ГБ	Ready	Нет	web-1	ru-central1-a	22.05.2026, в 22:06	—	fhmproud00mrurrd2m3d	—	...
☐ fhmvnhdtpo2obu7cnhv	Подключен	HDD	8 ГБ	Ready	Нет	replica	ru-central1-a	22.05.2026, в 22:03	—	fhmvnhdtpo2obu7cnhv	—	...
☐ disk-1	Подключен	HDD	1 ГБ	Ready	Нет	storage	ru-central1-a	22.05.2026, в 22:27	—	fhm8s2hpl1ehaq3rs25p	—	...
☐ disk-2	Подключен	HDD	1 ГБ	Ready	Нет	storage	ru-central1-a	22.05.2026, в 22:27	—	fhmqqp0ck3q01s15fipm	—	...
☐ disk-3	Подключен	HDD	1 ГБ	Ready	Нет	storage	ru-central1-a	22.05.2026, в 22:27	—	fhmhu4a66891beap2o14	—	...

Запушил в гит репозиторий

Задание 4

1. В файле `ansible.tf` создайте inventory-файл для ansible. Используйте функцию `templatefile` и файл-шаблон для создания ansible inventory-файла из лекции. Готовый код возьмите из демонстрации к лекции [demonstration2](#). Передайте в него в качестве переменных группы виртуальных машин из задания 2.1, 2.2 и 3.2, т. е. 5 VM.
2. Инвентарь должен содержать 3 группы и быть динамическим, т. е. обработать как группу из 2-х VM, так и 999 VM.
3. Добавьте в инвентарь переменную [fqdn](#).

```
1 [webservers]
2 web-1 ansible_host=<внешний ip-адрес> fqdn=<полное доменное имя виртуальной
  машины>
3 web-2 ansible_host=<внешний ip-адрес> fqdn=<полное доменное имя виртуальной
  машины>
4
5 [databases]
6 main ansible_host=<внешний ip-адрес> fqdn=<полное доменное имя виртуальной машины>
7 replica ansible_host=<внешний ip-адрес> fqdn=<полное доменное имя виртуальной
  машины>
8
9 [storage]
10 storage ansible_host=<внешний ip-адрес> fqdn=<полное доменное имя виртуальной
    машины>
```

Пример fqdn: `web1.ru-central1.internal` (в случае указания переменной hostname(не путать с переменной name)); `fhm8k1oojmm5lie8i22a.auto.internal` (в случае отсутствия переменной hostname - автоматическая генерация имени, зона изменяется на auto). нужную вам переменную найдите в документации провайдера или terraform console. 4. Выполните код. Приложите скриншот получившегося файла.

Разбор решения задачи 4

Создаю файл `ansible.tf` где используется `templatefile(...)` для генерации inventory (пример: <https://onhub.ru/d/18-sozdaem-inventory-dlya-ansible-s-pomoschyu-terraform>)

```
1 resource "local_file" "ansible_inventory" {
2     content = templatefile("${path.module}/hosts.tftpl", {
3         webserver = yandex_compute_instance.web
4         databases = values(yandex_compute_instance.db)
5         storage    = [yandex_compute_instance.storage]
6     })
7
8     filename = "${path.module}/inventory.ini"
9 }
```

- `yandex_compute_instance.web` при `count` уже является списком ресурсов, его можно передать как есть;

- `yandex_compute_instance.db` при `for_each` является `map`, а шаблону удобнее дать список, поэтому использую `values(...)`;
- `storage` - одиночный ресурс, но для единообразия заворачиваем его в список из одного элемента. Это позволяет в шаблоне везде работать одинаково через циклы.

Создаю файл `hosts.tftpl`

```
1 [webservers]
2 {% for host in webservers ~}
3   ${host.name} ansible_host=${host.network_interface[0].nat_ip_address}
   fqdn=${host.fqdn}
4 {% endfor ~}
5
6 [databases]
7 {% for host in databases ~}
8   ${host.name} ansible_host=${host.network_interface[0].nat_ip_address}
   fqdn=${host.fqdn}
9 {% endfor ~}
10
11 [storage]
12 {% for host in storage ~}
13   ${host.name} ansible_host=${host.network_interface[0].nat_ip_address}
   fqdn=${host.fqdn}
14 {% endfor ~}
```

Шаблон будет работать для любого количества машин потому, что строится циклом, а не ручным перечислением.

Поле `fqn` берётся напрямую из read-only атрибута ресурса `yandex_compute_instance`.

```
• vboxuser@KVA:~/terraform/hw-03$ terraform apply
yandex_vpc_network.develop: Refreshing state... [id=enp2fl93jhjibc9r98ov]
data.yandex_compute_image.ubuntu: Reading...
yandex_compute_disk.disk[2]: Refreshing state... [id=fhmbu4a66891beap2ol4]
yandex_compute_disk.disk[0]: Refreshing state... [id=fhm8s2hpl1ehaq3rs25p]
yandex_compute_disk.disk[1]: Refreshing state... [id=fhmqqp0ck3q01sl5fipm]
data.yandex_compute_image.ubuntu: Read complete after 0s [id=fd85f16aij1rvpce9gas]
yandex_vpc_subnet.develop: Refreshing state... [id=e9bv4rbjflbas3mbvqh8]
yandex_vpc_security_group.example: Refreshing state... [id=enpmttnpva47uq9lqaae]
yandex_compute_instance.db["replica"]: Refreshing state... [id=fhmeoe671732kedf7e9g]
yandex_compute_instance.db["main"]: Refreshing state... [id=fhmdjppq47damfeibel9m]
yandex_compute_instance.storage: Refreshing state... [id=fhm9utk3qpmsncc3rhhk]
yandex_compute_instance.web[0]: Refreshing state... [id=fhm5hr3sf0vbh6ele5qg]
yandex_compute_instance.web[1]: Refreshing state... [id=fhmpkpppm6crmpmfvqca]
```

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

```
# yandex_compute_instance.storage has changed
~ resource "yandex_compute_instance" "storage" {
  id          = "fhm9utk3qpmsncc3rhhk"
  + labels    = {}
  name       = "storage"
  # (14 unchanged attributes hidden)

  # (9 unchanged blocks hidden)
}
```

Unless you have made equivalent changes to your configuration, or ignored the relevant attributes using `ignore_changes`, the following plan may include actions to undo or respond to these changes.

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# local_file.ansible_inventory will be created
+ resource "local_file" "ansible_inventory" {
  + content = <<-EOT
    [webserver]
  }
```

```
web-1 ansible_host=89.169.143.159 fqdn=fhm5hr3sf0vbh6ele5qg.auto.internal
web-2 ansible_host=93.77.182.170 fqdn=fhmpkpppm6crmpmfvqca.auto.internal
```

```
[databases]
```

```
main ansible_host=84.252.131.99 fqdn=fhmdjpq47damfeibel9m.auto.internal
replica ansible_host=111.88.244.132 fqdn=fhmeoe671732kedf7e9g.auto.internal
```

```
[storage]
```

```
storage ansible_host=111.88.240.200 fqdn=fhm9utk3qpmsncc3rhhk.auto.internal
```

```
EOT
```

```
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5           = (known after apply)
+ content_sha1          = (known after apply)
+ content_sha256        = (known after apply)
+ content_sha512        = (known after apply)
+ directory_permission = "0777"
+ file_permission       = "0777"
+ filename              = "./inventory.ini"
+ id                    = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

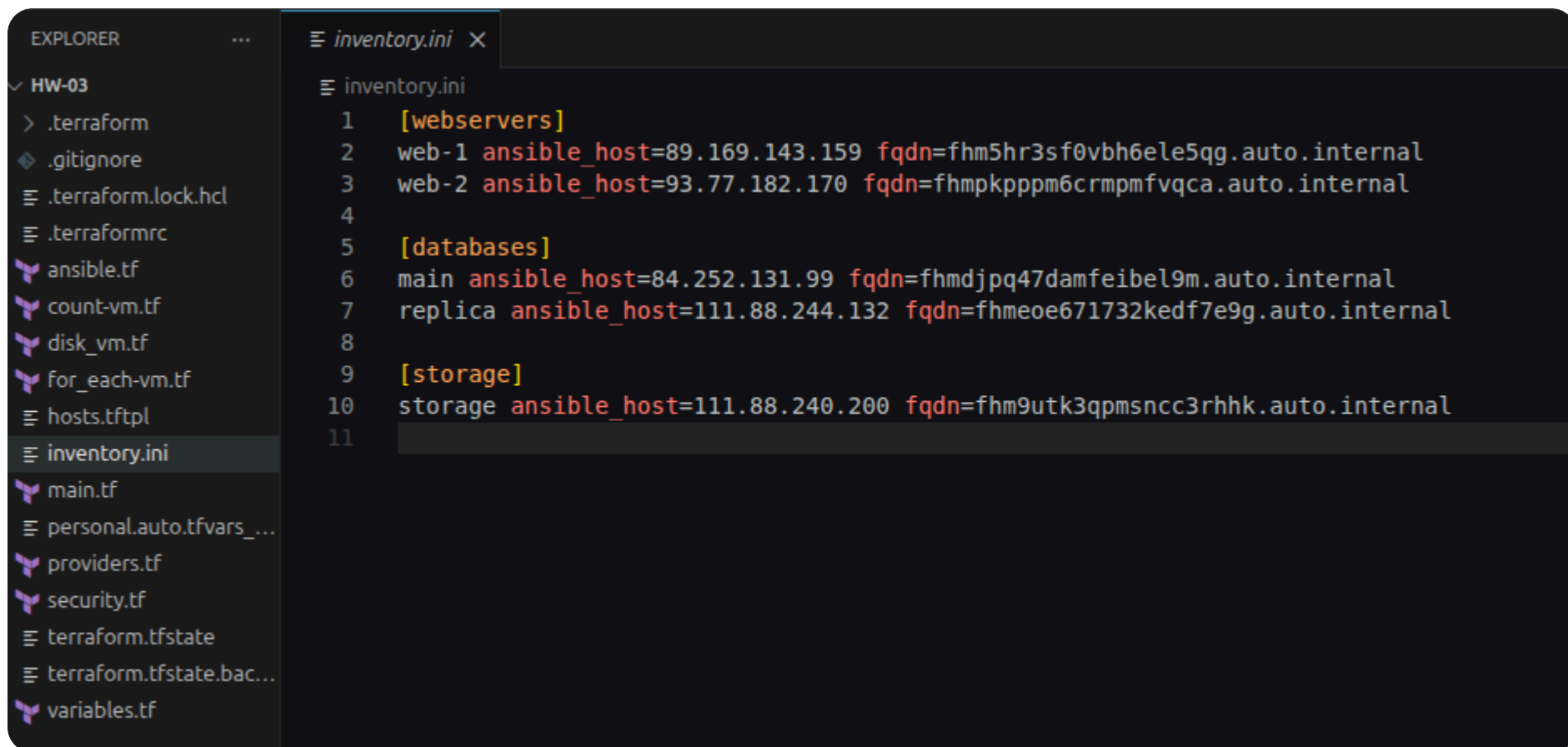
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.ansible_inventory: Creating...

local_file.ansible_inventory: Creation complete after 0s [id=13964a306201a36d2e55396389e633694301a686]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.



The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a file tree for a project named 'HW-03'. The files listed include '.terraform', '.gitignore', '.terraform.lock.hcl', '.terraformrc', 'ansible.tf', 'count-vm.tf', 'disk_vm.tf', 'for_each-vm.tf', 'hosts.tftpl', 'inventory.ini' (which is selected), 'main.tf', 'personal.auto.tfvars_...', 'providers.tf', 'security.tf', 'terraform.tfstate', 'terraform.tfstate.bac...', and 'variables.tf'. The main editor area displays the content of 'inventory.ini'. The file contains three groups: '[webserver]', '[databases]', and '[storage]'. Each group lists hosts with their 'ansible_host' and 'fqdn' values. Line numbers 1 through 11 are visible on the left of the editor area.

```
1  [webserver]
2  web-1 ansible_host=89.169.143.159 fqdn=fhm5hr3sf0vbh6ele5qg.auto.internal
3  web-2 ansible_host=93.77.182.170 fqdn=fhmpkpppm6crmpmfvqca.auto.internal
4
5  [databases]
6  main ansible_host=84.252.131.99 fqdn=fhmdjppq47damfeibel9m.auto.internal
7  replica ansible_host=111.88.244.132 fqdn=fhmeoe671732kedf7e9g.auto.internal
8
9  [storage]
10 storage ansible_host=111.88.240.200 fqdn=fhm9utk3qpmsncc3rhhk.auto.internal
11
```

Запустил в гит репозиторий

Дополнительные задания (со звездочкой*)

Задание 5* (необязательное)

1. Напишите output, который отобразит VM из ваших ресурсов count и for_each в виде списка словарей :

```
1  [  
2    {  
3      "name" = 'имя VM1'  
4      "id"   = 'идентификатор VM1'  
5      "fqdn" = 'Внутренний FQDN VM1'  
6    },  
7    {  
8      "name" = 'имя VM2'  
9      "id"   = 'идентификатор VM2'  
10     "fqdn" = 'Внутренний FQDN VM2'  
11   },  
12   ....  
13   ...итд любое количество VM в ресурсе(те требуется итерация по ресурсам, а не  
    хардкод) !!!!!!!!!!!!!!!!!!!!!!!  
14  ]
```

Приложите скриншот вывода команды `terraform output` .

Разбор решения задачи 5

создал файл `outputs.tf` следующего содержания:

```

1 output "vms_info" {
2     description = "Список ВМ из count и for_each ресурсов"
3     value = concat(
4         [
5             for vm in yandex_compute_instance.web : {
6                 name = vm.name
7                 id    = vm.id
8                 fqdn  = vm.fqdn
9             }
10        ],
11
12        [
13            for vm in values(yandex_compute_instance.db) : {
14                name = vm.name
15                id    = vm.id
16                fqdn  = vm.fqdn
17            }
18        ]
19    )
20 }

```

`concat(...)` встроенная функция Terraform, объединяет несколько списков в один. Без неё получились бы два отдельных output, вместо одного общего списка.

`for vm in yandex_compute_instance.web` итерация по списку ВМ из `count`. Для Terraform

`web[0]` и `web[1]` элементы списка.

`values(yandex_compute_instance.db)` : функция `values()` берёт map: `... {"main" = {...}, "replica" = {...}}...` и возвращает только значения в виде списка, отбрасывая ключи. Это нужно потому что `for_each` создаёт `map`, а `concat` работает со списками.

`{ name = vm.name, id = vm.id, fqdn = vm.fqdn }` - для каждой VM формирую объект (словарь) с тремя полями. `vm.fqdn` - это тот же read-only атрибут, что использован в `inventory`.


```
vboxuser@KVA:~/terraform/hw-03$ terraform apply
yandex_compute_disk.disk[2]: Refreshing state... [id=fhmbu4a66891beap2ol4]
yandex_vpc_network.develop: Refreshing state... [id=enp2fl93jhjibc9r98ov]
yandex_compute_disk.disk[0]: Refreshing state... [id=fhm8s2hpllehaq3rs25p]
yandex_compute_disk.disk[1]: Refreshing state... [id=fhmqqp0ck3q01sl5fipm]
data.yandex_compute_image.ubuntu: Read complete after 0s [id=fd85f16aijlrvpce9gas]
yandex_vpc_subnet.develop: Refreshing state... [id=e9bv4rbjflbas3mbvqh8]
yandex_vpc_security_group.example: Refreshing state... [id=enpmttnpva47uq9lqaee]
yandex_compute_instance.db["replica"]: Refreshing state... [id=fhmeoe671732kedf7e9g]
yandex_compute_instance.db["main"]: Refreshing state... [id=fhmdjppq47damfeibel9m]
yandex_compute_instance.storage: Refreshing state... [id=fhm9utk3qpmsncc3rhhk]
yandex_compute_instance.web[1]: Refreshing state... [id=fhmpkpppm6crmpmfvqca]
yandex_compute_instance.web[0]: Refreshing state... [id=fhm5hr3sf0vbh6ele5qg]
local_file.ansible_inventory: Refreshing state... [id=13964a306201a36d2e55396389e633694301a686]
```

Changes to Outputs:

```
+ vms_info = [
+ {
+   + fqdn = "fhm5hr3sf0vbh6ele5qg.auto.internal"
+   + id   = "fhm5hr3sf0vbh6ele5qg"
+   + name = "web-1"
+ },
+ {
+   + fqdn = "fhmpkpppm6crmpmfvqca.auto.internal"
+   + id   = "fhmpkpppm6crmpmfvqca"
+   + name = "web-2"
+ },
+ {
+   + fqdn = "fhmdjppq47damfeibel9m.auto.internal"
+   + id   = "fhmdjppq47damfeibel9m"
+   + name = "main"
+ },
+ {
+   + fqdn = "fhmeoe671732kedf7e9g.auto.internal"
+   + id   = "fhmeoe671732kedf7e9g"
+   + name = "replica"
+ },
+ ]
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

```

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

vms_info = [
  {
    "fqdn" = "fhm5hr3sf0vbh6ele5qg.auto.internal"
    "id" = "fhm5hr3sf0vbh6ele5qg"
    "name" = "web-1"
  },
  {
    "fqdn" = "fhmpkpppm6crmpmfvqca.auto.internal"
    "id" = "fhmpkpppm6crmpmfvqca"
    "name" = "web-2"
  },
  {
    "fqdn" = "fhmdjppq47damfeibel9m.auto.internal"
    "id" = "fhmdjppq47damfeibel9m"
    "name" = "main"
  },
  {
    "fqdn" = "fhmeoe671732kedf7e9g.auto.internal"
    "id" = "fhmeoe671732kedf7e9g"
    "name" = "replica"
  },
]
vboxuser@KVA:~/terraform/hw-03$

```

Запустил в гит репозиторий

Задание 7* (необязательное)

Ваш код возвращает вам следующий набор данных:

```
1 > local.vpc
2 {
3   "network_id" = "enp7i560tb28nageq0cc"
4   "subnet_ids" = [
```

```
5     "e9b0le401619ngf4h68n",
6     "e2lbar6u8b2ftd7f5hia",
7     "b0ca48coorjjq93u36pl",
8     "fl8ner8rjsio6rcpcf0h",
9 ]
10 "subnet_zones" = [
11     "ru-central1-a",
12     "ru-central1-b",
13     "ru-central1-c",
14     "ru-central1-d",
15 ]
16 }
```

Предложите выражение в terraform console, которое удалит из данной переменной 3 элемент из: subnet_ids и subnet_zones.(значения могут быть любыми) Образец конечного результата:

```
1 > <некое выражение>
2 {
3     "network_id" = "enp7i560tb28nageq0cc"
4     "subnet_ids" = [
5         "e9b0le401619ngf4h68n",
6         "e2lbar6u8b2ftd7f5hia",
7         "fl8ner8rjsio6rcpcf0h",
8     ]
```

```
9     "subnet_zones" = [  
10         "ru-central1-a",  
11         "ru-central1-b",  
12         "ru-central1-d",  
13     ]  
14 }
```

Разбор решения задания 7

Выражение для `terraform console`

```
1 merge(  
2     local.vpc,  
3     {  
4         subnet_ids    = concat(slice(local.vpc.subnet_ids, 0, 2),  
slice(local.vpc.subnet_ids, 3, length(local.vpc.subnet_ids)))  
5         subnet_zones = concat(slice(local.vpc.subnet_zones, 0, 2),  
slice(local.vpc.subnet_zones, 3, length(local.vpc.subnet_zones)))  
6     }  
7 )
```

- `merge(local.vpc, {...})` - `merge` объединяет два объекта. Если ключ совпадает, значение из второго объекта перезаписывает первый. Беру весь `local.vpc` и перезаписываю только `subnet_ids` и `subnet_zones`. `network_id` остаётся нетронутым.

- `slice(local.vpc.subnet_ids, 0, 2)` берёт элементы индекса от `0` до `1` включительно, то есть первые два: `["e9b0le401619ngf4h68n", "e2lbar6u8b2ftd7f5hia"]`.
- `slice(local.vpc.subnet_ids, 3, length(local.vpc.subnet_ids))` берёт элементы с индекса `3` до конца: `["fl8ner8rjsio6rcpcf0h"]`.
- `concat(..., ...)` склеивает два среза в один список, пропуская 3-й элемент (индекс `2`).

```
vboxuser@KVA:~/terraform/hw-03$ terraform console
> local.vpc
{
  "network_id" = "enp7i560tb28nageq0cc"
  "subnet_ids" = [
    "e9b0le401619ngf4h68n",
    "e2lbar6u8b2ftd7f5hia",
    "b0ca48coorjjq93u36pl",
    "fl8ner8rjsio6rcpcf0h",
  ]
  "subnet_zones" = [
    "ru-central1-a",
    "ru-central1-b",
    "ru-central1-c",
    "ru-central1-d",
  ]
}
> merge(local.vpc, { subnet_ids = concat(slice(local.vpc.subnet_ids, 0, 2), slice(local.vpc.subnet_ids, 3, length(local.vpc.subnet_ids))), subnet_zones = concat(slice(local.vpc.subnet_zones, 0, 2), slice(local.vpc.subnet_zones, 3, length(local.vpc.subnet_zones))) })
{
  "network_id" = "enp7i560tb28nageq0cc"
  "subnet_ids" = [
    "e9b0le401619ngf4h68n",
    "e2lbar6u8b2ftd7f5hia",
    "fl8ner8rjsio6rcpcf0h",
  ]
  "subnet_zones" = [
    "ru-central1-a",
    "ru-central1-b",
    "ru-central1-d",
  ]
}
```

Задание 8* (необязательное)

Идентифицируйте и устраните намеренно допущенную в tpl-шаблоне ошибку. Обратите внимание, что terraform сам сообщит на какой строке и в какой позиции ошибка!

```
1 [webservers]
2 %{~ for i in webservers ~}
3 ${i["name"]} ansible_host=${i["network_interface"][0]["nat_ip_address"]}
  platform_id=${i["platform_id "]}
4 %{~ endfor ~}
```

Пропущена закрывающая скобка в конце выражения `ansible_host : ["nat_ip_address"]}` <- тут ,
и ненужный пробел после id тут: `["platform_id "]`.